

StreamTN: A Low-Latency Streaming Chinese Text Normalization Model for Streaming TTS in Dialogue Systems

Wenhao Li¹, Jinrui Liang¹, Haoyu Zhang¹, Jingbin Hu¹, Xiaming Ren¹, Hanke Xie¹, Huakang Chen¹,
Chengyou Wang¹, Dake Guo¹, Linhan Ma¹, Su Feng², Houdun Liu², Yunxiang Chen², Lei Xie^{1,*}

¹Audio, Speech and Language Processing Group (ASLP@NPU),

Northwestern Polytechnical University, Xi'an, China

²Shenzhen Pime Technology Co., Ltd., Shenzhen, China

wenhao.li@mail.nwpu.edu.cn, lxie@nwpu.edu.cn

Abstract—Text-to-Speech (TTS) is an essential module that provides spoken responses in a spoken dialogue system (SDS) centered on a large language model (LLM). To ensure accurate TTS synthesis, responses generated by an LLM must be converted into TTS-readable formats via a Text Normalization (TN) module, imposing strict low-latency requirements in real-time SDS scenarios. Existing TN solutions are largely rule-based, rely on manual engineering, and generalize poorly to unseen patterns. Although an LLM itself can perform TN through prompt engineering, it faces key limitations: high first-token latency due to non-streaming processing, hallucination risks, and degraded intelligence or reasoning when the core LLM module is fine-tuned solely for TN. To address these challenges, we propose *StreamTN*, a lightweight LLM-based Chinese streaming TN model. Built on Qwen3-0.6B, StreamTN employs a dual-track streaming framework in which input tokens and output tokens are processed on two parallel tracks, enabling low-latency real-time inference without complex prompting. Moreover, task-specific fine-tuning yields superior TN performance and fewer hallucinations than rule-based systems and general-purpose LLMs. We also introduce a TN benchmark that spans diverse text scenarios, providing a comprehensive evaluation standard for speech generation in spoken dialogue systems. Experiments demonstrate the effectiveness of StreamTN in accuracy and inference latency.

Index Terms—text normalization, spoken dialogue systems, low-latency inference

I. INTRODUCTION

Spoken dialogue systems (SDS) have become an important interface for natural human–computer interaction, especially with the rapid development of large language models (LLMs). Modern LLM-centered SDS typically follow either an end-to-end paradigm, where speech understanding, response generation, and speech synthesis are jointly modeled, or a cascaded paradigm, where Automatic Speech Recognition (ASR), an LLM module, and Text-to-Speech (TTS) are organized as independent components [1]–[5]. Recent speech-language models further explore speech-native or real-time interaction with LLMs [6]–[9]. Although end-to-end systems have shown

promising progress, cascaded systems remain widely used in practical applications due to their controllability, modularity, ease of debugging, and flexibility in integrating external knowledge or retrieval modules.

In a cascaded SDS, the textual response produced by the LLM module serves as the direct input to the TTS module. However, LLM-generated responses often contain non-standard words (NSWs), such as numbers, dates, time expressions, phone numbers, units, chemical formulas, and mathematical expressions. These forms are readable to humans but can be ambiguous or unsuitable for direct speech synthesis. For example, a single number may correspond to different pronunciations depending on whether it appears in a date, a phone number, an amount, or a mathematical expression. Without proper text normalization (TN), such ambiguity may cause mispronunciations, unnatural prosody, or even synthesis failures in downstream TTS systems.

Text normalization has been extensively studied as the task of converting non-standard written expressions into context-appropriate verbalizations, with TTS being one of its primary applications. Traditional industrial TN systems often rely on handcrafted rules, regular expressions, and WFST-based grammars [10]–[12], which provide strong controllability but require substantial language-specific engineering. To reduce reliance on manually designed rules, neural approaches formulate TN as a sequence modeling task, predicting context-appropriate verbalizations from written input [13]. However, purely neural approaches may produce rare but severe unrecoverable errors, which is particularly problematic for TTS. For Mandarin TN, hybrid approaches have been proposed to combine rule-based systems with neural models such as multi-head self-attention [14]. More recently, FlatTN introduces a rule-guided flat-lattice Transformer to integrate expert rules into end-to-end Chinese TN and releases a large-scale Chinese TN dataset [15]. These studies significantly advance TN modeling and resource construction, but they mainly focus on offline or sentence-level normalization rather than low-latency streaming normalization in LLM-centered dialogue systems.

* indicates the corresponding author.

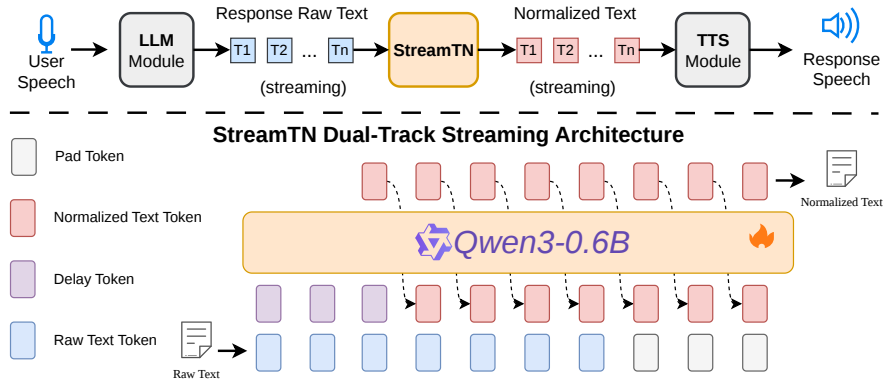


Fig. 1. The architecture of our proposed StreamTn model.

Recent progress in LLMs has further motivated prompt-based TN. PolyNorm, for example, explores few-shot LLM-based TN for TTS and introduces a multilingual benchmark covering diverse normalization phenomena [16]. This line of work demonstrates the potential of LLMs for scalable TN across languages. Nevertheless, directly applying prompt-based LLMs to cascaded SDS remains challenging because the TN module must process partial LLM outputs and provide normalized text to the TTS module in real time. Prompt-based normalization usually requires waiting for sufficient or complete context, introduces additional latency, and may still suffer from unstable formatting or hallucinated outputs. In contrast, our goal is not only to improve TN accuracy, but also to enable streaming input processing and streaming output generation with controllable first-packet delay.

A key challenge in LLM-centered cascaded SDS is that both the upstream LLM response and the downstream TTS synthesis are expected to operate in a streaming manner, especially as recent TTS systems increasingly emphasize scalable and low-latency streaming synthesis [17]. In this setting, the TN module is no longer a conventional offline text preprocessing component. Instead, it must incrementally convert partial LLM outputs into TTS-readable text while preserving pronunciation correctness and semantic consistency. Therefore, an effective TN module for dialogue TTS should satisfy two requirements simultaneously: it should generate accurate and TTS-readable normalized text, and it should do so incrementally with low latency.

This new application scenario also exposes a benchmark gap. Although evaluation methods for dialogue systems have been broadly studied [18], existing TN datasets and evaluations mainly focus on conventional NSW categories, offline sentence-level normalization, or multilingual TTS normalization, and they do not fully reflect the distribution, diversity, and latency requirements of real dialogue responses generated by LLMs. In particular, dialogue-oriented TTS must handle not only common expressions such as numbers, dates, time, phone numbers, and units, but also increasingly frequent complex outputs involving scientific terms, chemical formulas, and mathematical expressions. Without a scenario-aligned TN benchmark, it is difficult to systematically compare different

methods under the practical requirements of cascaded SDS, including normalization accuracy, robustness to diverse text patterns, and streaming latency.

To address these challenges, we propose StreamTN, a lightweight LLM-based Chinese streaming text normalization model for cascaded SDS. Built on Qwen3-0.6B [19], StreamTN introduces a dual-track streaming architecture that decouples raw-text input tokens and normalized-text output tokens into two parallel tracks. This design enables the model to begin normalization after receiving only a small number of input tokens, rather than waiting for the complete LLM response. Through task-specific fine-tuning, StreamTN learns structured normalization patterns while avoiding complex prompting and reducing hallucination risks.

In addition, we construct a Chinese dialogue-oriented TN benchmark tailored to streaming TTS in cascaded SDS. The benchmark covers 14 categories, including common NSW types such as numbers, dates, time, phone numbers, and units, as well as challenging scientific expressions such as chemical formulas and mathematical equations. It provides a scenario-aligned evaluation setting for measuring not only normalization accuracy, but also robustness across diverse dialogue outputs and latency under streaming inference.

The main contributions of this work are summarized as follows:

- We propose StreamTN, a dedicated Chinese streaming TN model with a dual-track architecture that enables incremental input processing and output generation with controllable first-packet delay.
- We construct a dialogue-oriented Chinese TN benchmark covering 14 categories of practical normalization scenarios, filling the gap of TN evaluation for streaming TTS in cascaded dialogue systems.
- Experiments show that StreamTN provides competitive normalization quality at low first-packet delay and improves further when additional context is available.
- To facilitate reproducibility and future research, we will release the trained StreamTN model and the accompanying benchmark upon publication.

A demonstration is available online.¹

II. METHOD

A. Overall architecture

As illustrated in Figure 1, StreamTN is inserted between the upstream LLM module and the downstream TTS module in a cascaded spoken dialogue system. The LLM module first generates a textual response from the user’s spoken query, and the TN module converts this response into a TTS-readable form before speech synthesis. In conventional cascaded systems, TN is usually performed by a rule-based module or a lightweight neural model after the complete textual response is available. StreamTN replaces this offline TN component and processes the LLM response incrementally, so that normalized text can be passed to the TTS module before the full response is finished [20].

This setting differs from standard sentence-level text normalization. In an offline TN system, the model can access the whole input sentence and use both left and right context to disambiguate non-standard words. In a streaming dialogue system, however, the input arrives token by token from the LLM module. The TN model therefore needs to decide whether the currently available prefix already contains enough information for normalization. Emitting too early may lead to incorrect readings, while waiting for too much context increases the first-packet latency of the TTS system. Thus, the architecture needs an explicit mechanism to control the amount of delayed context used for generation.

StreamTN is built on Qwen3-0.6B to reuse its general text modeling ability. The original model, however, follows a conventional autoregressive formulation in which input and output tokens are arranged along a single sequence. This form is not well suited to streaming TN, because output generation is coupled with the availability of the input sequence. Inspired by Qwen3-Omni [21] and delayed-stream modeling [22], we introduce a dual-track structure into the text model. The raw LLM output and the normalized text are represented on two separate tracks, and their embeddings are fused before being fed into the Transformer backbone.

With this structure, the input track provides the currently available raw-text context, while the output track carries the delayed normalized-text history. The model can therefore generate normalized tokens autoregressively while still consuming new raw tokens from the upstream LLM. When the LLM stream is still active, the model uses both tracks; when the raw input stream has ended, the input track is padded and generation continues based on the output history until the end-of-sequence token is produced. This formulation keeps the TN model compatible with standard autoregressive decoding, while allowing the amount of input lookahead to be controlled by a fixed delay parameter.

The following subsection gives the formal definition of the dual-track streaming process, including the construction of the

two tracks, representation fusion, the training objective, and first-packet delay.

B. Dual-track streaming formulation

Given a raw text sequence produced by the upstream LLM module, we denote the input token sequence as $\mathbf{x} = (x_1, x_2, \dots, x_n)$, where x_t denotes the raw-text token at time step t , and n is the input length. The corresponding normalized target sequence is denoted as $\mathbf{y} = (y_1, y_2, \dots, y_m)$, where y_i denotes the normalized-text token at output step i , and m is the target length. A conventional autoregressive text generation model typically learns the conditional distribution $p_\theta(\mathbf{y} | \mathbf{x})$, where θ denotes the model parameters. In offline text normalization, this formulation assumes that the complete input sequence is available before decoding begins. In the streaming setting considered here, however, the model only observes a prefix of the LLM response at each decoding step, making it necessary to reformulate the normalization process under partial input context.

Let d denote the token-delay parameter, which controls how many raw input tokens are observed before the first normalized token is emitted. For each aligned step t , we use $\bar{x}_t$ and $\bar{y}_t$ to denote the symbolic states on the raw-input track and the output-history track, respectively. The raw-input track is defined as

$$\bar{x}_t = \begin{cases} x_t, & 1 \leq t \leq n, \\ \langle \text{pad} \rangle, & t > n, \end{cases} \quad (1)$$

and the output-history track is delayed by d aligned steps:

$$\bar{y}_t = \begin{cases} \langle \text{delay} \rangle, & 1 \leq t \leq d, \\ y_{t-d}, & d < t \leq m + d, \\ \langle \text{pad} \rangle, & t > m + d. \end{cases} \quad (2)$$

Here, $\langle \text{delay} \rangle$ and $\langle \text{pad} \rangle$ are conceptual alignment symbols rather than learned vocabulary tokens. They have different semantic roles: the former indicates that output generation has not started, whereas the latter indicates that a track has ended. In the implementation, both symbols are realized as zero vectors after embedding. Let $\Phi(\cdot)$ denote the track-to-embedding mapping based on the token-embedding table $E(\cdot)$ inherited from Qwen3-0.6B:

$$\Phi(u) = E(u), \quad \Phi(\langle \text{delay} \rangle) = \Phi(\langle \text{pad} \rangle) = \mathbf{0}, \quad (3)$$

where u is an ordinary raw or normalized token. The two tracks share the same embedding space and are fused by element-wise addition:

$$\mathbf{z}_t = \Phi(\bar{x}_t) + \Phi(\bar{y}_t). \quad (4)$$

Let $T = \max(n, m + d)$ denote the aligned length. During training, the last fused position is removed so that every normalized target is predicted from its preceding causal state. The resulting sequence is processed by the Qwen3 Transformer with a causal attention mask:

$$\mathbf{h}_{1:T-1} = \text{Qwen3}_\theta(\mathbf{z}_{1:T-1}; \mathbf{M}_{\text{causal}}), \quad (5)$$

¹<https://supernova-neko.github.io/Stream-TN/>

where positional information is applied internally by Qwen3 according to the aligned position indices. Under this causal shift, the hidden state at aligned step $i + d - 1$ contains the raw prefix $x_{\leq \min(n, i+d-1)}$ and the normalized history $y_{< i}$, but not the current target y_i . The next-token distribution is therefore

$$p_{\theta}(y_i | x_{\leq \min(n, i+d-1)}, y_{< i}) = \text{softmax}(W_o \mathbf{h}_{i+d-1} + \mathbf{b}_o), \quad (6)$$

where W_o and $\mathbf{b}_o$ are the output projection matrix and bias, respectively.

This alignment gives the streaming factorization

$$p_{\theta}(\mathbf{y} | \mathbf{x}) = \prod_{i=1}^m p_{\theta}(y_i | x_{\leq \min(n, i+d-1)}, y_{< i}). \quad (7)$$

The delay parameter d determines the trade-off between latency and available input context. In particular, y_1 is predicted from $\mathbf{h}_d$ after the first d raw tokens have been observed. At the next aligned step, the model consumes the embedding of the previously generated token together with the next available raw-token embedding and predicts y_2 . Thus, no target token is included in the hidden state used to predict itself.

During training, supervision is applied only to valid normalized target tokens; zero-padded positions are excluded by the target mask. The training objective is the negative log-likelihood

$$\mathcal{L}_{\text{TN}} = - \sum_{i=1}^m \log p_{\theta}(y_i | x_{\leq \min(n, i+d-1)}, y_{< i}). \quad (8)$$

This objective trains the model to perform normalization under partial input context, rather than relying on the complete raw sequence.

During inference, StreamTN follows the same alignment using the autoregressive key-value cache. The model first receives d raw tokens and predicts $\hat{y}_1$ from $\mathbf{h}_d$. For $i > 1$, aligned step $t = i + d - 1$ combines the next raw-token embedding, when available, with $E(\hat{y}_{i-1})$, and the resulting hidden state predicts $\hat{y}_i$. After the raw stream ends, the raw-input state becomes $\langle \text{pad} \rangle$, whose mapped representation is $\mathbf{0}$, and decoding continues until an end-of-sequence token is produced or a predefined maximum generation budget is reached.

The implementation measures the model-side first-token computation time after the first d raw tokens have become available. In a complete spoken dialogue system, the end-to-end first-packet delay additionally includes the time required by the upstream LLM to produce these tokens:

$$\text{FPD}_{\text{e2e}}(d) = T_{\text{wait}}^{\text{LLM}}(d) + T_{\text{first}}^{\text{TN}}(d). \quad (9)$$

Here, $T_{\text{wait}}^{\text{LLM}}(d)$ denotes the upstream waiting time, whereas $T_{\text{first}}^{\text{TN}}(d)$ includes embedding the available prefix, the initial Transformer prefill, and computation of the first normalized-token logits. Reporting these two terms separately distinguishes intrinsic TN computation latency from the token-arrival rate of a particular upstream LLM.

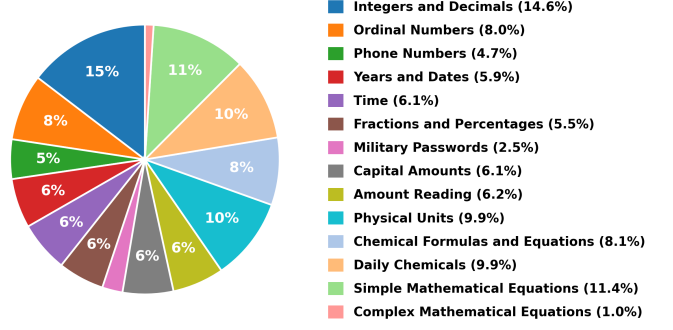


Fig. 2. Data distribution of text normalization categories.

C. Dataset construction

1) Taxonomy Design and Category Consolidation

Prior work, including FlatTN [15], organizes Chinese text normalization categories according to the Non-Standard Word (NSW) standard, which adopts a fine-grained classification scheme. However, modern LLM-based training requires a more concise data representation consisting of paired raw text and normalized text to support efficient manual inspection and model optimization [23]. Building on the taxonomy of FlatTN [15], the original categories are consolidated into ten unified types to simplify the label space while preserving coverage. In addition, to address user queries involving scientific content and the corresponding complex outputs generated by the LLM module, four new categories are introduced: simple chemical substances, complex chemical equations, simple mathematical formulas, and complex mathematical equations. Based on this refined taxonomy, an evaluation benchmark is constructed for Chinese SDS [24], [25].

2) Data Collection and Annotation Pipeline

After consolidating the taxonomy, we defined category-specific TN guidelines based on the pronunciation and readability requirements of Chinese TTS systems. The guidelines specify the expected normalization form for each category and were applied consistently during annotation and evaluation. We constructed the training corpus from two complementary sources. First, we screened publicly available Chinese text from DuReader [26], whose questions and supporting documents originate from real-world Baidu Search and Baidu Zhidao data. Regular-expression-based detectors were used to identify text containing normalizable NSW patterns, and only the matched samples were retained as raw inputs.

Second, because naturally occurring data provide limited coverage of the four newly introduced scientific categories—simple chemical substances, complex chemical equations, simple mathematical formulas, and complex mathematical equations—we used Qwen3-32B [19] to generate additional raw-text samples. These AI-generated samples were manually reviewed to remove unnatural expressions and obvious synthetic artifacts. Normalized targets for both the filtered open-

source samples and the AI-generated samples were produced using DeepSeek-R1-70B [27] under the same predefined TN specifications. To assess annotation quality, we randomly sampled 5% of the training set for manual verification. Of the inspected samples, 98.2% were judged consistent with the predefined TN guidelines. In addition, all 1,262 held-out test references were manually inspected and corrected according to the same guidelines.

III. EXPERIMENTS

A. Experimental setup

1) Datasets

A comprehensive dataset covering diverse TN categories has been constructed, comprising 95,793 training samples and 1,262 held-out test samples. The training set is used for model optimization, whereas the test set evaluates generalization to unseen and challenging TN scenarios.

2) Baselines

To comprehensively evaluate StreamTN across different modeling paradigms, we compare it with four representative baselines:

- **WeTextProcessing** [28]: a widely used open-source rule-based toolkit covering common TN patterns such as numerals, dates, and units. We retain its default rule set.
- **FlatTN** [15]: a rule-guided end-to-end Chinese TN model that integrates expert rules and is evaluated on the same test set.
- **BiLSTM** [29], [30]: a fully data-driven lightweight encoder-decoder that learns normalization mappings directly from paired data without handcrafted rules.
- **Qwen3-0.6B** [19]: a general-purpose pre-trained language model evaluated with task-oriented prompting, but without task-specific fine-tuning or streaming adaptation.

3) Evaluation Metrics

Model performance is evaluated using character-level micro-averaged precision (Micro-P), recall (Micro-R), and F1 (Micro-F1). For each prediction-reference pair, we first obtain a minimum-cost Levenshtein alignment and count character matches (M), substitutions (S), deletions (D), and insertions (I). Following the implementation used in our evaluation, matches are treated as true positives, while substitutions contribute to both false positives and false negatives; insertions contribute to false positives and deletions to false negatives. The alignment counts are accumulated over the entire evaluation set before computing the metrics, rather than averaging sentence-level scores. Accordingly, the three primary metrics are defined as

$$P_{\text{micro}} = \frac{\sum_i M_i}{\sum_i (M_i + S_i + I_i)}, \quad (10)$$

$$R_{\text{micro}} = \frac{\sum_i M_i}{\sum_i (M_i + S_i + D_i)}.$$

$$F_{1,\text{micro}} = \frac{2P_{\text{micro}}R_{\text{micro}}}{P_{\text{micro}} + R_{\text{micro}}}. \quad (11)$$

This corpus-level aggregation gives each aligned character equal weight and is used for both the overall comparison and the category-level analysis.

4) Experimental Environment

The model uses PyTorch with Hugging Face Transformers. Unless noted, main experiments use full-parameter fine-tuning. For ablation, we also examine parameter-efficient fine-tuning via LoRA, which adds low-rank adapters to the attention projections (q_proj , k_proj , v_proj , o_proj) while freezing backbone weights. In LoRA, the rank is 8, the scaling factor is 32, and the dropout is 0.05. Training runs on four NVIDIA A6000 GPUs with dynamic batching. The learning rate starts at 1×10^{-5} , decaying to 1×10^{-6} using cosine annealing over 5,000 steps. To account for variability arising from training initialization, each trainable model configuration is independently trained with five random seeds, and its normalization metrics are reported as mean $\pm$ standard deviation. Deterministic rule-based and off-the-shelf baselines are evaluated once. StreamTN inference uses greedy decoding on a single NVIDIA A6000 GPU. For end-to-end first-packet-delay measurements, the upstream Qwen3-32B [19] is served with vLLM using tensor parallelism on two A6000 GPUs and generates approximately 20 tokens/s; its tokens are streamed to StreamTN as soon as they become available. The reported delay follows Eq. (9) and includes both the waiting time for the first d upstream tokens and the StreamTN computation required to produce the first normalized token.

B. Model performance

1) Overall Performance

Table I reports the Micro-F1 and Micro-Precision of all methods. The rule-based WeTextProcessing baseline performs reasonably on predefined patterns, but its limited rule coverage restricts its ability to handle diverse and context-dependent expressions. FlatTN obtains a Micro-F1 of 0.7569, compared with 0.7853 for WeTextProcessing. The general-purpose Qwen3-0.6B baseline performs substantially worse than the task-specific models. Even with task-oriented prompting, its outputs remain susceptible to format instability and hallucination. More elaborate prompts can improve normalization quality, but they also increase prefill cost and first-token latency, making this approach less suitable for real-time dialogue.

At the selected four-frame delay, StreamTN achieves a Micro-F1 of 0.8937 ± 0.0009 , which is comparable to the BiLSTM result of 0.8941 ± 0.0015 , while obtaining a higher Micro-Precision (0.8931 ± 0.0022 versus 0.8677 ± 0.0012). This operating point is chosen to balance normalization quality and first-packet delay rather than to maximize offline accuracy. As shown in Table IV, allowing more input context further improves StreamTN and enables it to surpass BiLSTM in Micro-F1. Overall, StreamTN provides a stronger practical trade-off than the evaluated baselines by combining competitive normalization performance with incremental processing and controllable first-packet latency.

TABLE I
MICRO-F1 AND MICRO-PRECISION OF DIFFERENT MODELS

Method	Micro-F1	Micro-P
WeTextProcessing [28]	0.7853	0.7586
Qwen3-0.6B [19]	0.4872	0.5282
FlatTN [15]	0.7569 ± 0.0019	0.7390 ± 0.0017
BiLSTM [29]	0.8941 ± 0.0015	0.8677 ± 0.0012
StreamTN	0.8937 ± 0.0009	0.8931 ± 0.0022

2) Model Performance Across Different Categories

Table II reports the micro-averaged precision, recall, and F1-score of StreamTN across 14 TN categories. StreamTN performs best on physical units, capital amounts, and time expressions, achieving Micro-F1 scores of 0.977, 0.975, and 0.967, respectively. Strong results are also obtained for integers and decimals, fractions and percentages, and years and dates, all of which exceed 0.95 Micro-F1. These categories generally follow regular and frequently observed normalization patterns. In contrast, complex mathematical equations and chemical formulas remain the most challenging categories, with Micro-F1 scores of 0.750 and 0.791, respectively. Their specialized notation, long verbalizations, and structural diversity increase the difficulty of character-level normalization. Overall, the small standard deviations indicate that StreamTN performs consistently across training seeds.

TABLE II
MICRO PRECISION, RECALL, AND F1 SCORES OF DIFFERENT TN TYPES

TN Type	Micro-P	Micro-R	Micro-F1
Integers and Decimals	$0.959 \pm .006$	$0.963 \pm .004$	$0.961 \pm .004$
Ordinal Numbers	$0.923 \pm .010$	$0.929 \pm .005$	$0.926 \pm .007$
Phone Numbers	$0.919 \pm .008$	$0.917 \pm .008$	$0.918 \pm .008$
Years and Dates	$0.949 \pm .009$	$0.957 \pm .009$	$0.953 \pm .009$
Time	$0.956 \pm .002$	$0.978 \pm .003$	$0.967 \pm .001$
Fractions and Percentages	$0.964 \pm .009$	$0.947 \pm .006$	$0.955 \pm .007$
Military Passwords	$0.925 \pm .003$	$0.924 \pm .004$	$0.925 \pm .003$
Capital Amounts	$0.970 \pm .004$	$0.979 \pm .001$	$0.975 \pm .002$
Amount Reading	$0.927 \pm .006$	$0.919 \pm .011$	$0.923 \pm .007$
Physical Units	$0.976 \pm .003$	$0.978 \pm .003$	$0.977 \pm .003$
Chemical Formulas	$0.785 \pm .008$	$0.797 \pm .006$	$0.791 \pm .006$
Daily Chemicals	$0.913 \pm .008$	$0.884 \pm .009$	$0.898 \pm .007$
Simple Math Equations	$0.956 \pm .005$	$0.943 \pm .005$	$0.950 \pm .005$
Complex Math Equations	$0.749 \pm .014$	$0.751 \pm .007$	$0.750 \pm .009$

C. Ablation Study

1) Impact of LoRA Fine-Tuning and System Prompt

Table III evaluates LoRA-based parameter-efficient fine-tuning and system-prompt conditioning within the StreamTN framework. The full StreamTN model achieves the best results, with a Micro-F1 of 0.8937 ± 0.0009 and a Micro-Precision of 0.8931 ± 0.0022 . Replacing full-parameter fine-tuning with LoRA reduces Micro-F1 to 0.6335 ± 0.0012 and Micro-Precision to 0.6250 ± 0.0015 , indicating that updating only low-rank adapters is insufficient to learn the precise transformations required by TN. Adding a system prompt also yields a small but consistent decrease, producing a Micro-F1 of 0.8852 ± 0.0018 and a Micro-Precision of 0.8824 ± 0.0021 . Overall, the ablation results demonstrate the importance of full-parameter fine-tuning and show that StreamTN can generate structured normalized text without carefully engineered prompts.

TABLE III
RESULTS OF ABLATION EXPERIMENT.

Method	Micro-F1	Micro-P
StreamTN	0.8937 ± 0.0009	0.8931 ± 0.0022
w/ LoRA Fine-tuning	0.6335 ± 0.0012	0.6250 ± 0.0015
w/ System Prompt	0.8852 ± 0.0018	0.8824 ± 0.0021

2) Impact of Streaming Strategy

Table IV reports the performance of StreamTN under different delay-frame configurations. The non-streaming model achieves the highest Micro-F1 (0.9639) and Micro-Precision (0.9620) because it has access to the complete input sequence, but it cannot produce normalized output until the entire sequence has been received. Within the streaming framework, increasing the delay allows the model to observe more contextual information before normalization, thereby resolving ambiguities in context-dependent expressions and substantially improving normalization performance. Specifically, as the delay increases from 1 to 16 frames, Micro-F1 rises from 0.7030 to 0.9239 and Micro-Precision from 0.7135 to 0.9303. This improvement, however, is accompanied by an increase in FPD from 75 ms to 756 ms. We therefore adopt the 4-frame configuration in the main experiments as a practical trade-off between normalization quality and responsiveness; it achieves a Micro-F1 of 0.8937 and a Micro-Precision of 0.8931 with an FPD of 213 ms.

TABLE IV
STREAMTN PERFORMANCE UNDER DIFFERENT DELAY FRAMES. FPD (MS) DENOTES FIRST PACKET DELAY IN MILLISECONDS. THE CONFIGURATION MARKED WITH * INDICATES THE SETTING ADOPTED IN OUR MAIN EXPERIMENTS.

Strategy	Delay frame	Micro-F1	Micro-P	FPD (ms)
Non-Streaming	-	0.9639 ± 0.0011	0.9620 ± 0.0012	-
Streaming	1 frame	0.7030 ± 0.0010	0.7135 ± 0.0018	75
	2 frames	0.7952 ± 0.0015	0.8062 ± 0.0026	121
	4 frames*	0.8937 ± 0.0009	0.8931 ± 0.0022	213
	8 frames	0.9072 ± 0.0011	0.9093 ± 0.0020	394
	16 frames	0.9239 ± 0.0008	0.9303 ± 0.0015	756

IV. CONCLUSIONS

This paper presented StreamTN, a Chinese text normalization model designed for low-latency cascaded spoken dialogue systems. StreamTN combines task-specific fine-tuning with a dual-track architecture that processes incoming text and normalized output on separate tracks, allowing normalization to proceed before the complete input is available. On our benchmark, the four-frame configuration achieved a Micro-F1 of 0.8937 with a first-packet delay of 213 ms. Its normalization performance was comparable to the BiLSTM baseline while supporting incremental processing, and additional input context produced further improvements at the cost of higher latency. The category-level results show that most errors occur in structurally complex mathematical and chemical expressions. We will release the model and benchmark to support further work on streaming text normalization. Future work will focus on these difficult categories and extend StreamTN to additional languages and application domains.

REFERENCES

- [1] T. Wen, D. Vandyke, N. Mrksic, M. Gasic, L. M. Rojas-Barahona, P. Su, S. Ultes, and S. J. Young, “A network-based end-to-end trainable task-oriented dialogue system,” in *Proceedings of the 15th Conference of the European Chapter of the Association for Computational Linguistics: Volume 1, Long Papers*, 2017, pp. 438–449.
- [2] S. J. Young, M. Gasic, B. Thomson, and J. D. Williams, “POMDP-based statistical spoken dialog systems: A review,” *Proceedings of the IEEE*, vol. 101, no. 5, pp. 1160–1179, 2013.
- [3] Z. Zhang, R. Takanobu, Q. Zhu, M. Huang, and X. Zhu, “Recent advances and challenges in task-oriented dialog systems,” *Science China Technological Sciences*, vol. 63, no. 10, pp. 2011–2027, 2020.
- [4] S. Ji, Y. Chen, M. Fang, J. Zuo, J. Lu, H. Wang, Z. Jiang, L. Zhou, S. Liu, X. Cheng *et al.*, “WavChat: A survey of spoken dialogue models,” *CoRR*, vol. abs/2411.13577, 2024.
- [5] W. Cui, D. Yu, X. Jiao, Z. Meng, G. Zhang, Q. Wang, S. Y. Guo, and I. King, “Recent advances in speech language models: A survey,” in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2025, pp. 13 943–13 970.
- [6] D. Zhang, S. Li, X. Zhang, J. Zhan, P. Wang, Y. Zhou, and X. Qiu, “SpeechGPT: Empowering large language models with intrinsic cross-modal conversational abilities,” in *Findings of the Association for Computational Linguistics: EMNLP 2023*, 2023, pp. 15 757–15 773.
- [7] A. Défossez, L. Mazaré, M. Orsini, A. Royer, P. Pérez, H. Jégou, E. Grave, and N. Zeghidour, “Moshi: a speech-text foundation model for real-time dialogue,” *CoRR*, vol. abs/2410.00037, 2024.
- [8] Z. Xie and C. Wu, “Mini-Omni: Language models can hear, talk while thinking in streaming,” *CoRR*, vol. abs/2408.16725, 2024.
- [9] Q. Fang, S. Guo, Y. Zhou, Z. Ma, S. Zhang, and Y. Feng, “LLaMA-Omni: Seamless speech interaction with large language models,” in *International Conference on Learning Representations*, vol. 2025, 2025, pp. 57 607–57 624.
- [10] K. Gorman, “Pynini: A Python library for weighted finite-state grammar compilation,” in *Proceedings of the SIGFSM Workshop on Statistical NLP and Weighted Automata*, 2016, pp. 75–80.
- [11] P. Ebdon and R. Sproat, “The Kestrel TTS text normalization system,” *Natural Language Engineering*, vol. 21, no. 3, pp. 333–353, 2015.
- [12] NVIDIA, “NeMo text processing: Text normalization and inverse text normalization toolkit,” <https://github.com/NVIDIA/NeMo-text-processing>, 2023, GitHub repository.
- [13] H. Zhang, R. Sproat, A. H. Ng, F. Stahlberg, X. Peng, K. Gorman, and B. Roark, “Neural models of text normalization for speech applications,” *Computational Linguistics*, vol. 45, no. 2, pp. 293–337, 2019.
- [14] J. Zhang, J. Pan, X. Yin, C. Li, S. Liu, Y. Zhang, Y. Wang, and Z. Ma, “A hybrid text normalization system using multi-head self-attention for Mandarin,” in *2020 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2020, pp. 6694–6698.
- [15] W. Dai, C. Song, X. Li, Z. Wu, H. Pan, X. Li, and H. Meng, “An end-to-end Chinese text normalization model based on rule-guided flat-lattice transformer,” in *2022 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2022, pp. 7122–7126.
- [16] M. Wong, A. Alshehri, S. Kao, and H. He, “PolyNorm: Few-shot LLM-based text normalization for text-to-speech,” in *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing: Industry Track*, 2025, pp. 77–85.
- [17] Z. Du, Y. Wang, Q. Chen, X. Shi, X. Lv, T. Zhao, Z. Gao, Y. Yang, C. Gao, H. Wang *et al.*, “CosyVoice 2: Scalable streaming speech synthesis with large language models,” *CoRR*, vol. abs/2412.10117, 2024.
- [18] J. Deriu, Á. Rodrigo, A. Otegi, G. Echegoyen, S. Rosset, E. Agirre, and M. Cieliebak, “Survey on evaluation methods for dialogue systems,” *Artificial Intelligence Review*, vol. 54, no. 1, pp. 755–810, 2021.
- [19] A. Yang, A. Li, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Gao, C. Huang, C. Lv *et al.*, “Qwen3 technical report,” *CoRR*, vol. abs/2505.09388, 2025.
- [20] S. Arora, J. Tian, H. Futami, J. Jung, J. Shi, Y. Kashiwagi, E. Tsunoo, and S. Watanabe, “Chain-of-thought training for open E2E spoken dialogue systems,” in *Proceedings of the 26th Annual Conference of the International Speech Communication Association (INTERSPEECH)*, 2025, pp. 4833–4837.
- [21] J. Xu, Z. Guo, H. Hu, Y. Chu, X. Wang, J. He, Y. Wang, X. Shi, T. He, X. Zhu *et al.*, “Qwen3-Omni technical report,” *CoRR*, vol. abs/2509.17765, 2025.
- [22] N. Zeghidour, E. Kharitonov, M. Orsini, V. Volhejn, G. de Marmiesse, E. Grave, P. Pérez, L. Mazaré, and A. Défossez, “Streaming sequence-to-sequence learning with delayed streams modeling,” *CoRR*, vol. abs/2509.08753, 2025.
- [23] S. Zhang, L. Dong, X. Li, S. Zhang, X. Sun, S. Wang, J. Li, R. Hu, T. Zhang, G. Wang *et al.*, “Instruction tuning for large language models: A survey,” *ACM Computing Surveys*, vol. 58, no. 7, pp. 1–36, 2026.
- [24] D. Kiela, M. Bartolo, Y. Nie, D. Kaushik, A. Geiger, Z. Wu, B. Vidgen, G. Prasad, A. Singh, P. Ringshia *et al.*, “Dynabench: Rethinking benchmarking in NLP,” in *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, 2021, pp. 4110–4124.
- [25] Y. Huang, Y. Bai, Z. Zhu, J. Zhang, J. Zhang, T. Su, J. Liu, C. Lv, Y. Zhang, Y. Fu *et al.*, “C-Eval: A multi-level multi-discipline Chinese evaluation suite for foundation models,” *Advances in Neural Information Processing Systems*, vol. 36, pp. 62 991–63 010, 2023.
- [26] W. He, K. Liu, J. Liu, Y. Lyu, S. Zhao, X. Xiao, Y. Liu, Y. Wang, H. Wu, Q. She *et al.*, “DuReader: A Chinese machine reading comprehension dataset from real-world applications,” in *Proceedings of the Workshop on Machine Reading for Question Answering*, 2018, pp. 37–46.
- [27] DeepSeek-AI, “DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning,” *CoRR*, vol. abs/2501.12948, 2025.
- [28] Z. Peng *et al.*, “WeTextProcessing: A text normalization toolkit,” <https://github.com/wenet-e2e/WeTextProcessing>, 2023, GitHub repository.
- [29] M. Sunkara, C. Shivade, S. Bodapati, and K. Kirchhoff, “Neural inverse text normalization,” in *2021 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2021, pp. 7573–7577.
- [30] R. Sproat and N. Jaitly, “An RNN model of text normalization,” in *Proceedings of the 18th Annual Conference of the International Speech Communication Association (INTERSPEECH)*, 2017, pp. 754–758.